

Grafana Observability Setup Guide - Complete POC

Overview

This document provides a complete step-by-step guide to set up Grafana dashboards with email and webhook alerting for a Spring Boot observability application. The setup includes monitoring for high latency, errors, and database incidents.

Table of Contents

1. [Prerequisites](#)
 2. [Project Structure](#)
 3. [Spring Boot Configuration](#)
 4. [Docker Setup with Grafana LGTM Stack](#)
 5. [Swagger API Documentation](#)
 6. [Grafana Dashboard Configuration](#)
 7. [Email Alert Setup](#)
 8. [Webhook Alert Setup](#)
 9. [Testing and Validation](#)
 10. [Troubleshooting](#)
-

Prerequisites

- Java 21+
 - Maven 3.8+
 - Docker & Docker Compose
 - Spring Boot 4.1.0
 - Grafana v13.1.1+
-

Project Structure

```
observability/
├── src/
│   ├── main/
│   │   ├── java/com/observability/dashboard/
│   │   │   ├── controller/
│   │   │   │   └── OrderController.java
│   │   │   ├── service/
│   │   │   │   └── OrderService.java
│   │   │   ├── entity/
│   │   │   │   └── Order.java
│   │   │   ├── config/
│   │   │   │   ├── SwaggerConfig.java
│   │   │   └── ObservabilityApplication.java
│   └── resources/
```

```
| | | application.yaml
| | | logback-spring.xml
| compose.yaml
| pom.xml
| Grafana-Observability-Setup-Guide.md
```

Spring Boot Configuration

1. Dependencies (pom.xml)

```
<dependencies>
  <!-- Spring Boot Starters -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-opentelemetry</artifactId>
  </dependency>

  <!-- OpenTelemetry -->
  <dependency>
    <groupId>io.opentelemetry.instrumentation</groupId>
    <artifactId>opentelemetry-logback-appender-1.0</artifactId>
    <version>2.21.0-alpha</version>
  </dependency>

  <!-- Swagger/OpenAPI -->
  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.1.0</version>
  </dependency>

  <!-- Metrics -->
  <dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
  </dependency>
</dependencies>
```

2. Application Configuration (application.yaml)

```
spring:
  application:
    name: observability

# Swagger/OpenAPI Configuration
springdoc:
  swagger-ui:
    path: /swagger-ui.html
    enabled: true
  api-docs:
    path: /v3/api-docs

# OpenTelemetry & Monitoring Configuration
management:
  otlp:
    metrics:
      export:
        url: http://localhost:4318/v1/metrics
  opentelemetry:
    tracing:
      export:
        otlp:
          endpoint: http://localhost:4318/v1/traces
    logging:
      export:
        otlp:
          endpoint: http://localhost:4318/v1/logs
  tracing:
    sampling:
      probability: 1.0

server:
  shutdown: immediate

logging:
  level:
    org.springframework.boot.actuator.autoconfigure.opentelemetry: DEBUG
```

3. Controller with Monitoring Endpoints

```
@RestController
@RequestMapping("/api/orders")
@Tag(name = "Order Controller", description = "Endpoints for order management and monitoring incidents")
public class OrderController {

    @GetMapping
    @Operation(summary = "Get all orders", description = "Retrieve a list of all orders (normal operation)")
    public List<Order> getOrders() {
```

```

        return orderService.getOrders();
    }

    @GetMapping("/high-latency")
    @Operation(summary = "High latency test", description = "Test endpoint that
simulates high latency for monitoring")
    public String highLatency() throws InterruptedException {
        return orderService.highLatency(); // Simulates 2-3 second delay
    }

    @GetMapping("/error")
    @Operation(summary = "Error test", description = "Test endpoint that throws
RuntimeException for error monitoring")
    public String error() {
        return orderService.error(); // Throws RuntimeException
    }

    @GetMapping("/database-error")
    @Operation(summary = "Database error test", description = "Test endpoint that
simulates database connection failure")
    public String databaseError() {
        return orderService.databaseError(); // Simulates DB connection failure
    }
}

```

Docker Setup with Grafana LGTM Stack

Docker Compose Configuration (compose.yaml)

```

services:
  grafana-lgtm:
    image: 'grafana/otel-lgtm:latest'
    ports:
      - '3000:3000' # Grafana UI
      - '4317'      # OTLP gRPC
      - '4318'      # OTLP HTTP
    environment:
      # SMTP Configuration for Email Alerts
      - GF_SMTP_ENABLED=true
      - GF_SMTP_HOST=smtp.gmail.com:587
      - GF_SMTP_USER=your-email@gmail.com
      - GF_SMTP_PASSWORD=your-gmail-app-password
      - GF_SMTP_FROM_ADDRESS=your-email@gmail.com
      - GF_SMTP_FROM_NAME=Grafana Observability
      - GF_SMTP_SKIP_VERIFY=false
      - GF_SMTP_EHLO_IDENTITY=localhost

```

Start the Stack

```
# Start Grafana LGTM stack
docker-compose -f compose.yaml up -d

# Start Spring Boot application
mvn spring-boot:run
```

Access Points

- **Grafana Dashboard:** <http://localhost:3000> (admin/admin)
- **Spring Boot API:** <http://localhost:8080/api/orders>
- **Swagger UI:** <http://localhost:8080/swagger-ui.html>
- **Prometheus Metrics:** <http://localhost:8080/actuator/prometheus>

Swagger API Documentation

Swagger Configuration

```
@Configuration
public class SwaggerConfig {
    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Observability Dashboard API")
                .version("1.0.0")
                .description("Spring Boot Observability Application with
monitoring endpoints"));
    }
}
```

Test Endpoints

1. **Normal Operation:** `GET /api/orders`
2. **High Latency:** `GET /api/orders/high-latency` (2-3 second delay)
3. **Error Simulation:** `GET /api/orders/error` (throws RuntimeException)
4. **Database Error:** `GET /api/orders/database-error` (simulates DB failure)

Grafana Dashboard Configuration

1. Access Grafana

- Navigate to: <http://localhost:3000>
- Login: admin/admin (change password on first login)

2. Configure Prometheus Data Source

1. Go to **Connections** → **Data sources**
2. Click **"Add data source"**
3. Select **Prometheus**
4. URL: `http://localhost:9090`
5. Click **"Save & test"**

3. Create Performance Dashboard

Panel 1: Request Rate

```
sum(rate(http_server_requests_total{uri=~"/api/orders.*"}[5m])) by (uri)
```

Panel 2: Response Time (95th Percentile)

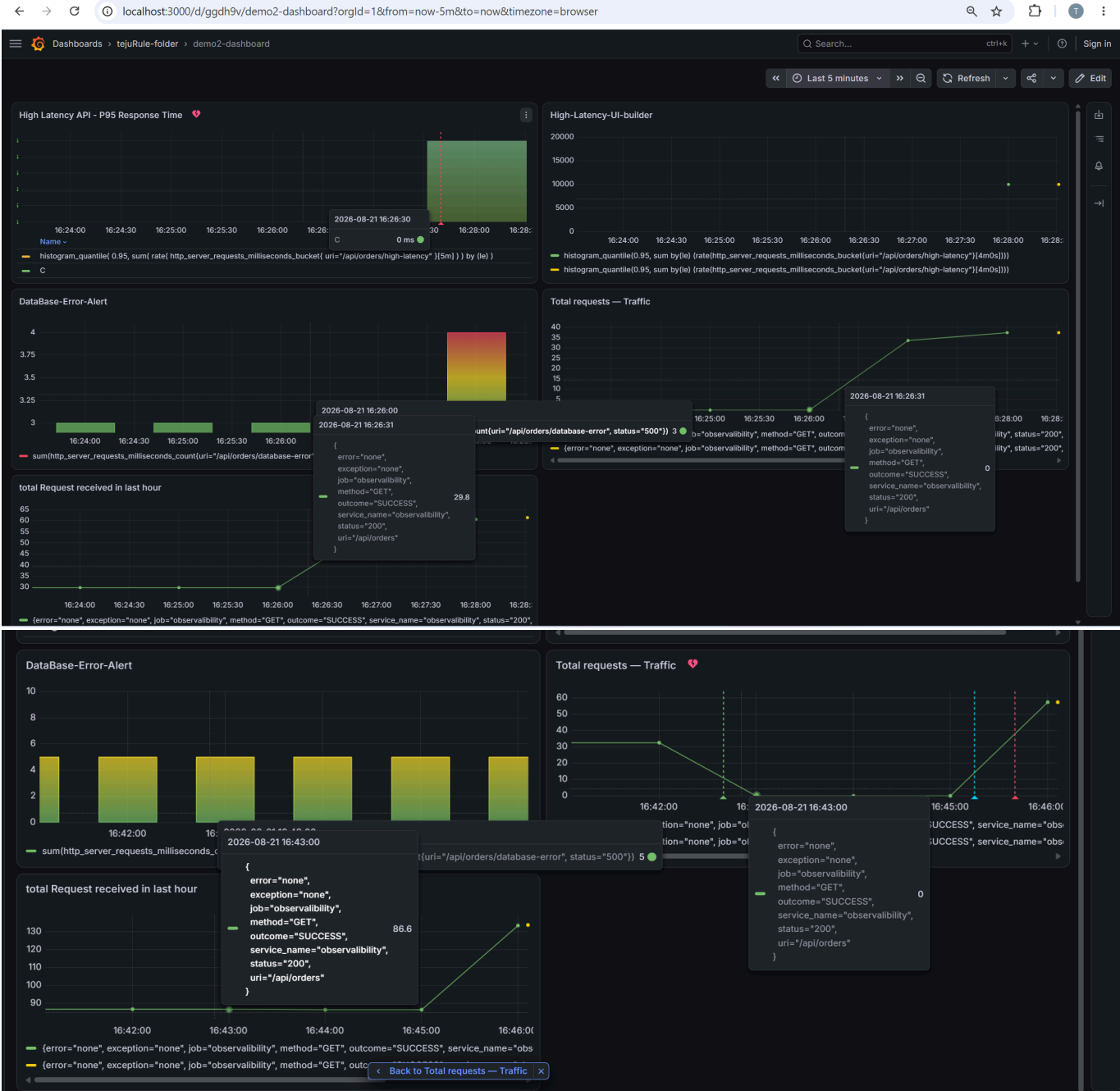
```
histogram_quantile(0.95,  
  sum(rate(http_server_requests_duration_seconds_bucket{uri=~"/api/orders.*"}  
[5m]))  
  by (le, uri)  
)
```

Panel 3: Error Rate by Endpoint

```
sum(rate(http_server_requests_total{uri=~"/api/orders.*", status=~"[45].."}[5m]))  
by (uri, status)
```

Panel 4: Active Requests

```
http_server_active_requests{uri=~"/api/orders.*"}
```





SpringBootApplication-Grafana-RuleAlert › Total requests — Traffic

🔥 1 firing instances

FiringTotal requests — TrafficView alert

Values

A=57.49688558536412 C=1

Labels

alertname	Total requests — Traffic
error	none
exception	none
grafana_folder	SpringBootApplication-Grafana-RuleAlert
job	observability
method	GET
outcome	SUCCESS
service_name	observability
status	200
uri	/api/orders

SilenceView dashboardView panel

Observed 35s before this notification was delivered, at 2026-08-21 11:15:40 +0000 UTC

Email Alert Setup

1. Gmail SMTP Configuration

Get Gmail App Password:

1. Enable 2-Factor Authentication in Google Account
2. Go to **Security** → **App passwords**
3. Generate app password for "Grafana"
4. Use the 16-character password in Docker configuration

Update compose.yaml:

```
environment:  
  - GF_SMTP_USER=your-actual-email@gmail.com
```


- GF SMTP_PASSWORD=your-16-char-app-password
- GF SMTP_FROM_ADDRESS=your-actual-email@gmail.com

2. Create Contact Point

1. Go to **Alerting** → **Contact points**
2. Click **"Add contact point"**
3. **Name:** `Email Alerts`
4. **Contact point type:** `Email`
5. **Addresses:** Enter your email address
6. Click **"Test"** to verify configuration
7. **Save**

3. Create Alert Rules

High Latency Alert

1. **Alerting** → **Alert rules** → **"New rule"**

Query Configuration:

```
# Query A: 95th percentile latency
histogram_quantile(0.95,
  sum(rate(http_server_requests_duration_seconds_bucket{uri=~"/api/orders.*"}
[5m]))
  by (le, uri)
)

# Expression B: Threshold
# Operation: Threshold
# Input: A
# Is above: 0.5
# Make this the alert condition: ✓
```

Alert Details:

- **Rule name:** `High Latency Alert - Orders API`
- **Folder:** `Observability Alerts`
- **Evaluate every:** `1m`
- **Evaluate for:** `1m`
- **Summary:** `High latency detected on {{ $labels.uri }}`
- **Description:** `95th percentile latency is {{ $value }}s on endpoint {{ $labels.uri }}`

Error Rate Alert

Query Configuration:

```
# Query A: Error count
increase(http_server_requests_total{
  uri=~"/api/orders.*(error|database-error)",
  status="500"
}[1m])

# Expression B: Threshold > 0
```

Alert Details:

- **Rule name:** Error Alert - Orders API
- **Evaluate every:** 30s
- **Evaluate for:** 0s
- **Summary:** Errors detected on Orders API
- **Description:** {{ \$value }} errors detected in the last minute on {{ \$labels.uri }}

4. Configure Notification Policies

1. **Alerting** → **Notification policies**
2. **Edit** default policy or create new nested policy
3. **Contact point:** Select Email Alerts
4. **Grouping:** alertname, uri
5. **Timing:**
 - Group wait: 10s
 - Group interval: 5m
 - Repeat interval: 4h

Webhook Alert Setup

1. Webhook Options

Option A: Slack Webhook

1. **Create Slack App:**
 - Go to <https://api.slack.com/apps>
 - Create new app → From scratch
 - Enable **Incoming Webhooks**
 - Add webhook to workspace
 - Copy webhook URL
2. **Grafana Configuration:**
 - **Contact point type:** Slack
 - **Webhook URL:** Paste your Slack webhook URL
 - **Channel:** #alerts
 - **Title:** Grafana Alert: {{ .GroupLabels.alertname }}

Option B: Discord Webhook

1. Create Discord Webhook:

- Server Settings → Integrations → Webhooks
- Create webhook, copy URL

2. Grafana Configuration:

- **Contact point type:** `Discord`
- **Webhook URL:** Paste Discord webhook URL

Option C: Generic Webhook (Testing)

1. Use Webhook.site for Testing:

- Go to <https://webhook.site>
- Copy your unique URL

2. Grafana Configuration:

- **Contact point type:** `Webhook`
- **URL:** Paste webhook.site URL
- **HTTP Method:** `POST`

2. Custom Webhook Payload

```
{
  "alerts": [
    {
      "status": "{{ .Status }}",
      "labels": {
        "alertname": "{{ .GroupLabels.alertname }}",
        "service": "observability",
        "severity": "{{ .CommonLabels.severity }}"
      },
      "annotations": {
        "summary": "{{ .CommonAnnotations.summary }}",
        "description": "{{ .CommonAnnotations.description }}"
      },
      "generatorURL": "{{ .GeneratorURL }}",
      "fingerprint": "{{ .Fingerprint }}"
    }
  ],
  "groupKey": "{{ .GroupKey }}",
  "status": "{{ .Status }}",
  "receiver": "webhook-alerts",
  "groupLabels": {{ .GroupLabels | toJson }},
  "commonLabels": {{ .CommonLabels | toJson }},
  "commonAnnotations": {{ .CommonAnnotations | toJson }}
```

```
"externalURL": "{{ .ExternalURL }}"
}
```

Testing and Validation

1. Generate Test Traffic

```
# Test normal endpoint
curl http://localhost:8080/api/orders

# Generate high latency alert
curl http://localhost:8080/api/orders/high-latency

# Generate error alert
curl http://localhost:8080/api/orders/error

# Generate database error
curl http://localhost:8080/api/orders/database-error
```

2. Monitor Alert Status

1. Check Alert Rules: Alerting → Alert rules

- Status should change from **Normal** → **Pending** → **Alerting**

2. View Active Alerts: Alerting → Alert groups

- See currently firing alerts

3. Check Notification History: Alerting → Notification history

- Verify alerts were sent successfully

3. Validate Metrics in Explore

```
# Check if metrics are being collected
http_server_requests_total{uri="/api/orders/high-latency"}

# Verify latency metrics
http_server_requests_duration_seconds{uri="/api/orders/high-latency"}

# Check error metrics
http_server_requests_total{uri="/api/orders/error", status="500"}
```

4. Dashboard Verification

Create dashboard panels to visualize:

- Request volume over time
 - Response time percentiles (50th, 95th, 99th)
 - Error rate trends
 - Alert status indicators
-

Troubleshooting

Common Issues and Solutions

1. SMTP Not Configured Error

Error: SMTP not configured, check your grafana.ini config file's [smtp] section

Solution:

- Update `compose.yaml` with correct SMTP settings
- Restart Grafana container: `docker-compose restart`
- Test contact point after restart

2. No Data in Metrics

Error: Alert shows "No data"

Solutions:

- Verify application is running: `curl http://localhost:8080/actuator/health`
- Check Prometheus metrics: `curl http://localhost:8080/actuator/prometheus | grep http_server`
- Test queries in Grafana Explore before creating alerts
- Ensure correct metric names in queries

3. Alerts Not Triggering

Possible Causes:

- Notification policy not configured
- Contact point not working
- Alert condition threshold too high
- Evaluation period too short

Debug Steps:

1. Check alert rule status in UI
2. Test contact point manually
3. Lower threshold temporarily for testing
4. Verify notification policy routing

4. Swagger UI 404 Error

Solution:

- Ensure SpringDoc dependency is included
- Verify `springdoc.swagger-ui.path` in `application.yaml`
- Access correct URL: `http://localhost:8080/swagger-ui.html`

5. Docker Container Issues

```
# Check container status
docker ps

# View Grafana logs
docker logs observability-grafana-lgtm-1

# Restart services
docker-compose -f compose.yaml down
docker-compose -f compose.yaml up -d
```

Advanced Configuration

1. Multi-Channel Alerting

Configure both email and webhook for critical alerts:

1. Create multiple contact points
2. Use notification policies with label matchers
3. Route different severity levels to different channels

2. Alert Silencing

Temporarily disable alerts during maintenance:

- **Alerting** → **Silences** → **"New silence"**
- Set matchers and duration

3. Custom Alert Templates

Create custom email/webhook templates:

- **Alerting** → **Notification templates**
- Use Go templating syntax for dynamic content

Summary

This POC demonstrates a complete observability setup with:

- ✓ **Spring Boot Application** with monitoring endpoints
- ✓ **Grafana LGTM Stack** via Docker
- ✓ **Swagger API Documentation**
- ✓ **Prometheus Metrics Collection**

- ✓ **Email Alerts** via SMTP
- ✓ **Webhook Alerts** for Slack/Discord
- ✓ **Dashboard Visualization**
- ✓ **Alert Rule Configuration**
- ✓ **Comprehensive Testing Strategy**

The setup provides real-time monitoring for:

- **High Latency** (>500ms response time)
- **Error Rates** (HTTP 500 errors)
- **Database Issues** (connection failures)
- **Service Health** (uptime monitoring)

All configuration is containerized and easily reproducible across environments.